

Gen-AI training pre-work and pre-read

Overview	1
Pre-requisite.....	1
Activate your VS Code virtual environment.....	2
Understanding Project Root folder.....	2
Understanding Virtual Environment	2
Activate the Virtual Environment	3
Python essentials.....	4
Using functions in python.....	4
Working with data tables using pandas	4
Learn to use .env for API keys and private configuration	4
Working with data files	5
Using matplotlib for data visualization and analysis	5
Accessing and consuming services using APIs.....	5
Gen AI Basic app.....	5
Google Colab and Kaggle.....	7
Create your first colab notebook.....	7
Working with secret keys in Colab.....	8
Working with data files in Colab	10
Git	13

Overview

This document describes the pre-work and pre-read for the participants attending the Gen AI training, so they get a head-start on it.

It is expected to take 2 to 4 hours to go through the below steps depending on your existing familiarity with these topics.

Pre-requisite

The setup of required software such as VS Code, Git etc. must already be done on the laptop. If not, please refer to the following document for the same:

<https://github.com/vijay-agrawal/genaitraining/blob/main/training-materials/0.1%20Gen%20AI%20Labs%20Setup.pdf>

Activate your VS Code virtual environment

Understanding Project Root folder

Project root folder (such as **genaitraining**) or any folder in which you are putting your code, provides the base folder to work against.

When opening VS Code, this is the folder you will open (not any other subfolder)

This is important as your project virtual environment, private configuration (.env) will all reside in your project root folder.

Hence, whenever starting VS Code, **always open the project root folder and not any other folder**

Understanding Virtual Environment

Virtual environment facilitates creation of a separate folder where all your python dependencies are saved. This ensures that you do not have to reinstall any library everytime and also that the right versions of your libraries are available to your application code (no conflicts with any other project or setup on your machine)

Following video explains the concept and benefits of virtual environments:

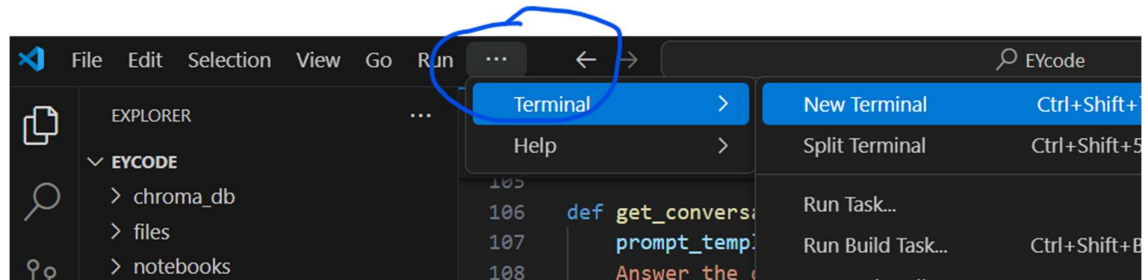
<https://youtu.be/KxvKCSwlUv8?si=6HQHCEJ2wkXnJ0QX>

Follow along the video below or steps outlined below to create a virtual environment

<https://youtu.be/GZbeL5AcTgw?si=cvCzwnbiXPZEWFMA>

1. Create a new folder for your project, called **genaitraining (or code)** in your home directory or in Desktop (this folder should already exist if the setup has been done). If not create it by following the setup document:
2. Open the folder in VS Code (File > Open Folder)

3. Start a terminal



Activate the Virtual Environment

If the setup has already been done, you should have the project base folder and a .venv or venv folder inside it already present. If not, follow the instructions in the setup document to set it up:

<https://github.com/vijay-agrawal/genaitraining/blob/main/training-materials/0.1%20Gen%20AI%20Labs%20Setup.pdf>

Activate the virtual environment with the following command:

venv\Scripts\activate

You will see a popup asking you to use this virtual environment for your project. Click on “Yes”

Make sure your virtual environment is active (you should see the venv name in the command prompt)

```
14 "AZURE_OPENAI_ENDPOINT": os.getenv("AZURE_OPENAI_ENDPOINT"),
15 "AZURE_OPENAI_MODEL_DEPLOYMENT_NAME": os.getenv("AZURE_OPENAI_MODEL_DEPLO
16 "AZURE_OPENAI_MODEL_NAME": os.getenv("AZURE_OPENAI_MODEL_NAME"),
17 "AZURE_OPENAI_API_KEY": os.getenv("AZURE_OPENAI_API_KEY"),
18 "AZURE_OPENAI_API_VERSION": os.getenv("AZURE_OPENAI_API_VERSION")

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Downloading certifi-2024.8.30-py3-none-any.whl (167 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading h11-0.14.0-py3-none-any.whl (58 kB)
Installing collected packages: typing-extensions, sniffio, jiter, idna, h11, distro, co
ydantic-core, httpcore, anyio, pydantic, httpx, openai
Successfully installed annotated-types-0.7.0 anyio-4.6.0 certifi-2024.8.30 colorama-0.4
httpx-0.27.2 idna-3.10 jiter-0.6.1 openai-1.51.2 pydantic-2.9.2 pydantic-core-2.23.4 s
s-4.12.2
PS C:\Users\BN743PF\Downloads\EYcode\EYcode> python -m venv venv
PS C:\Users\BN743PF\Downloads\EYcode\EYcode> venv\Scripts\activate
(venv) PS C:\Users\BN743PF\Downloads\EYcode\EYcode>
```

Python essentials

Download the python_basicapps.zip from:

https://github.com/vijay-agrawal/genaitraining/blob/main/code/00python_basicapps.zip

Unzip it into the **genaitraining** or your project root folder

It contains simple python scripts and a datafile **loanapproval_hist_data.csv**.

Open it in Excel and examine the data.

Loanapproved is the target column (value we want to teach the machine to predict) and other columns are features that influence the target.

Using functions in python

The demo functions_demo.py can be reviewed to understand how function calling, parameter passing is done in Python

Working with data tables using pandas

<https://www.youtube.com/watch?v=tRKeLrwfUgU>

Review the pandas demo code present in pandas_demo.py.

Run it by pressing the play button on top right.

Edit it to play with various dataframe capabilities such as sorting, adding a new derived column etc. Use help of your favourite AI chatbot to generate the code for the same and test it out.

Learn to use .env for API keys and private configuration

Best practice in python based application development is to keep private configuration parameters such as API Keys, passwords etc in a **.env** file

When your python application loads it can read the private key value pairs from the .env file and use them.

This gives flexibility and security and is a common practice to deal with private configuration parameters.

See the video below on how to setup a .env file for your project:

https://www.youtube.com/watch?v=8dlQ_nDE7dQ

Open the dotenv_demo.py in VS Code

Run it by using the “Play” button on top right in your VS Code.

Alternatively, you can open the terminal, activate your virtual environment

➤ Venv\scripts\activate

And issue following command:

➤ python dotenv_demo.py

The script should run and print the environment variables present in your .env file

Working with data files

Data files such as csv files can be easily loaded in Python. They are usually converted to pandas dataframe for further analysis.

Review the file reading demo code present in dataloading.py.

Run it by pressing the play button on top right.

You can check the data file it is loading, present in the data folder.

Using matplotlib for data visualization and analysis

Exploratory Data Analytics (EDA) with python

Run the data_visualization.py to see how matplotlib library helps with data visualization and analysis

A good video on how to do EDA with python:

<https://www.youtube.com/watch?v=xi0vhXFPegw>

Also search on Kaggle for existing notebooks people have made on the same.

<https://www.kaggle.com/code>

Accessing and consuming services using APIs

Review and run the code weather_api_json.py

It uses Weather API service from openweathermap (You can register to get your own free key)

Above should give a good handle on Python essentials for this program

Gen AI Basic app

You will now test a very basic Gen AI Python app, that will use Azure Open AI Keys to access an Open AI LLM hosted on Azure cloud.

Unzip the following into your project root folder:

https://github.com/vijay-agrawal/genaitraining/blob/main/code/01azure_openai.zip

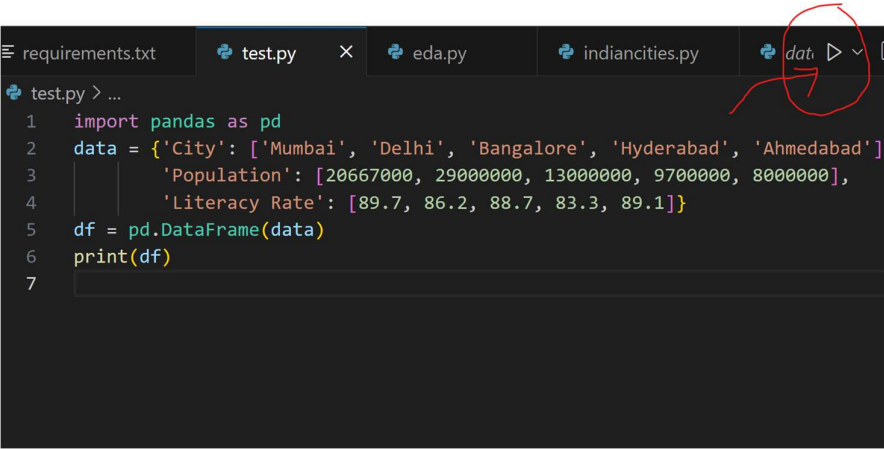
Unzip its contents into the genaitraining (or code) folder

Do not unzip into venv (Note that the virtual environment folder (.venv or venv) should not be touched once created)

The access keys are emailed separately as .env.zip file. Unzip it inside your project root folder.

Make sure you have .env file in your project root folder and it contains the Azure Open AI access keys.

Run the **test_azure_openai.py** by selecting it in the left pane and clicking on the **Play button** or by giving the command in terminal



```
test.py > ...
1 import pandas as pd
2 data = {'City': ['Mumbai', 'Delhi', 'Bangalore', 'Hyderabad', 'Ahmedabad'],
3         'Population': [20667000, 29000000, 13000000, 9700000, 8000000],
4         'Literacy Rate': [89.7, 86.2, 88.7, 83.3, 89.1]}
5 df = pd.DataFrame(data)
6 print(df)
7
```

If unable to run, you may need to select Python Interpreter:

- Press Ctrl+Shift+P (Cmd+Shift+P on macOS)
- Type "Python: Select Interpreter"
- Choose the interpreter from your virtual environment

Also ensure your virtual environment is enabled in the terminal.

In VS Code, open terminal and type the following if not enabled

➤ `venv\scripts\activate`

You should see the output properly showing the response from the LLM

Try to give different prompts such as “Who is the Prime Minister of India” or “Who is the Chief Minister of Maharashtra” or “Tell me about latest results of TCS” etc.

You are now ready to explore the exciting world of AI Apps in the classroom!

Google Colab and Kaggle

Google colab and Kaggle (<https://kaggle.com>) provide a cloud based Python development environment using **Jupyter notebook** concept.

Notebook makes it easy to write and execute python code inline. Below are steps outlined for colab. You can do similar with Kaggle if colab is not workable for you. Kaggle also provides many pre-built notebooks by contributors that are helpful for learning.

Create your first colab notebook

Get started with Google colab by going to:

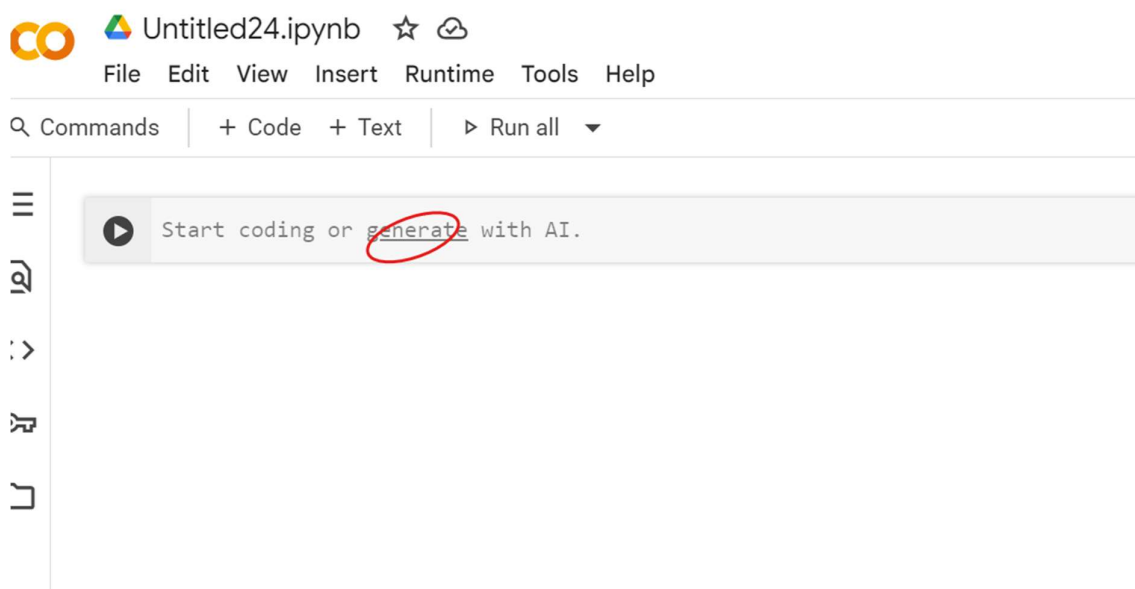
<https://colab.research.google.com/> and registering with your personal gmail account.

Create your first notebook by following the video at:

<https://www.youtube.com/watch?v=RLYoEylHL6A>

Try to put some of the python code from the python demos above into colab and execute them.

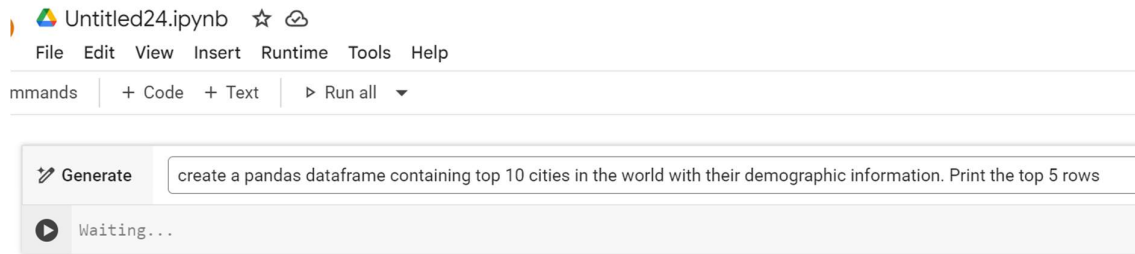
Google colab now also has **code generation** integrated into it!



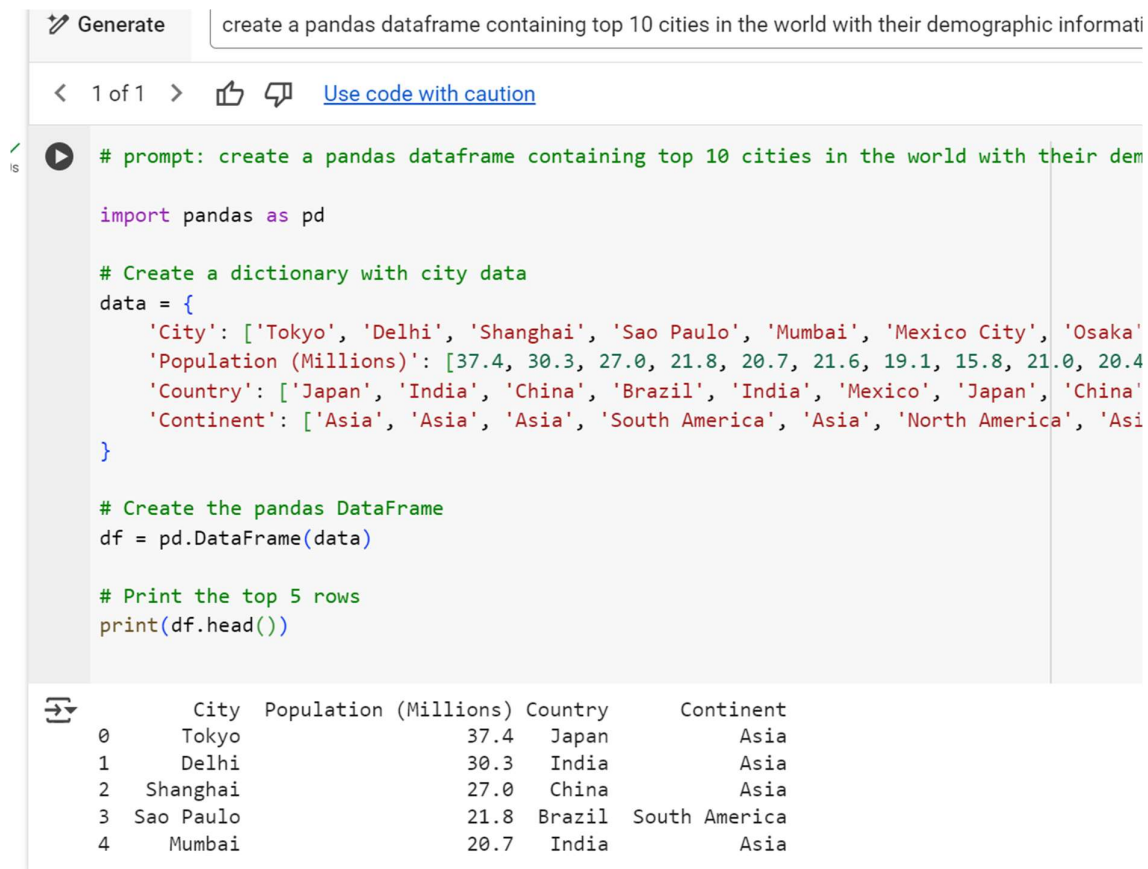
Click on “generate” and you can generate any code you want by just giving the prompt.

For example:

create a pandas dataframe containing top 10 cities in the world with their demographic information. Print the top 5 rows



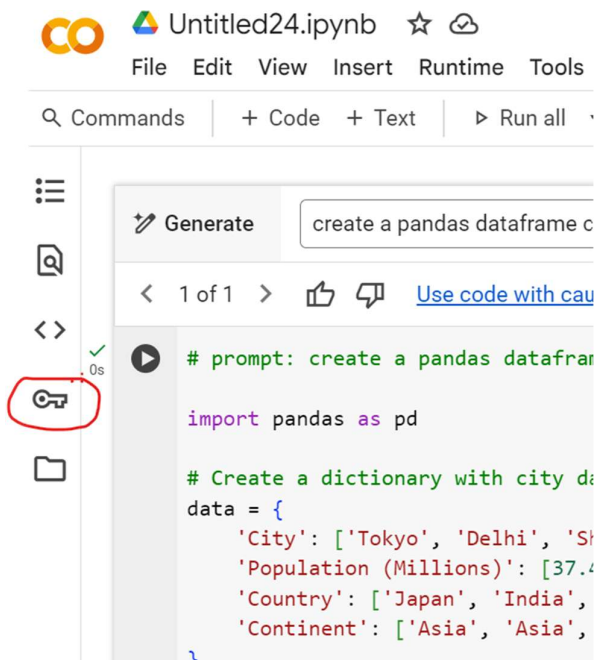
Click on the “Play” button and it will generate the code



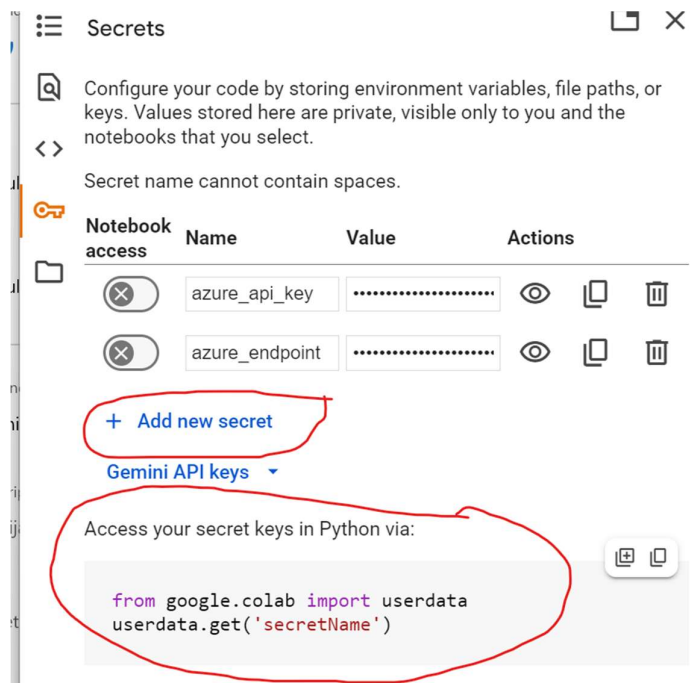
Working with secret keys in Colab

Secret Keys are a way to keep your private configuration information such as API keys secure. (It is similar to the .env concept in your python project in VS Code)

Click on the “Key” icon:



You can add new KEY just like you did in .env file



It shows the code how to use them in your notebook

Copy that code and paste it in your notebook

EXERCISE:

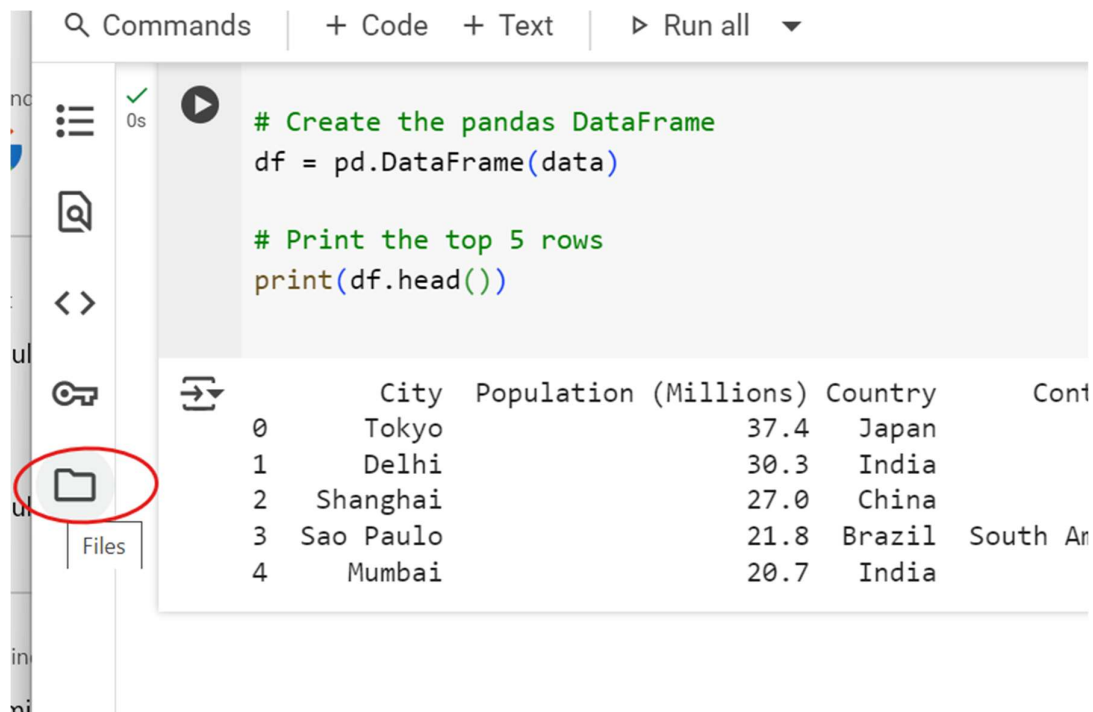
Take the code from the `weather_api_json.py` and make it work in colab.

Hint: remove the `load_dotenv()` and replace with `colabs' userdata.get()` after setting the `OPENWEATHERMAP_API_KEY` secret key in Colab

Working with data files in Colab

Colab offers a filesystem where you can upload your data files and use them in the notebook.

Click on the folder icon on the left



The screenshot shows the Google Colab interface. On the left, the 'Files' sidebar is visible, with a folder icon circled in red. The main area displays a code cell with the following Python code:

```
# Create the pandas DataFrame
df = pd.DataFrame(data)

# Print the top 5 rows
print(df.head())
```

Below the code, the output of the `df.head()` command is shown as a table:

	City	Population (Millions)	Country	Cont
0	Tokyo	37.4	Japan	
1	Delhi	30.3	India	
2	Shanghai	27.0	China	
3	Sao Paulo	21.8	Brazil	South Ar
4	Mumbai	20.7	India	

Click on the “Upload” icon to upload a data file to session storage

There is a data file in the `python_basicapps` under `data` folder, named `loanapproval_hist_data.csv` – you can upload that.

The screenshot shows the Google Colab interface. At the top, the title bar says "Untitled24.ipynb" with a star and a cloud icon. Below it is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A toolbar contains "Commands", "+ Code", "+ Text", and "Run all". The left sidebar is titled "Files" and shows a file upload icon circled in red with a tooltip that says "Upload to session storage". Below this is a folder named "sample_data". The main area shows a code cell with the following Python code:

```
# Create the pandas DataFrame
df = pd.DataFrame(data)

# Print the top 5 rows
print(df.head())
```

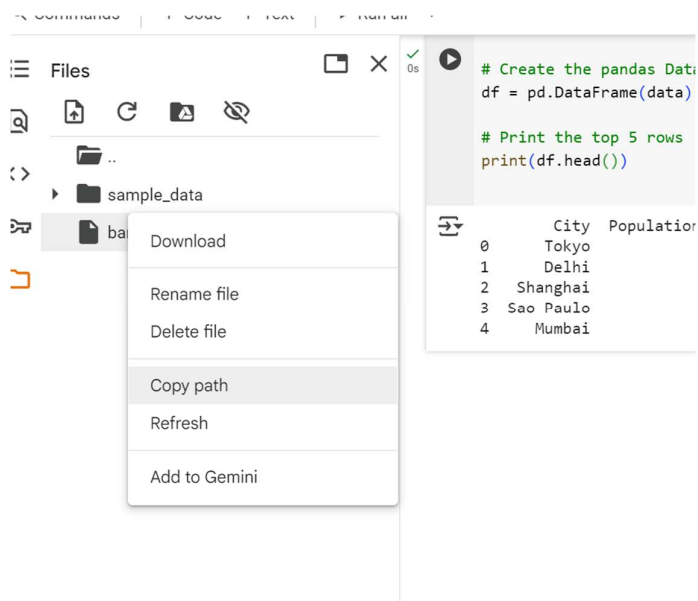
The code cell has a status bar showing a green checkmark and "0s". Below the code, the output is displayed as a table:

	City	Populati
0	Tokyo	
1	Delhi	
2	Shanghai	
3	Sao Paulo	
4	Mumbai	

Once the file is uploaded it will show up in the left pane.

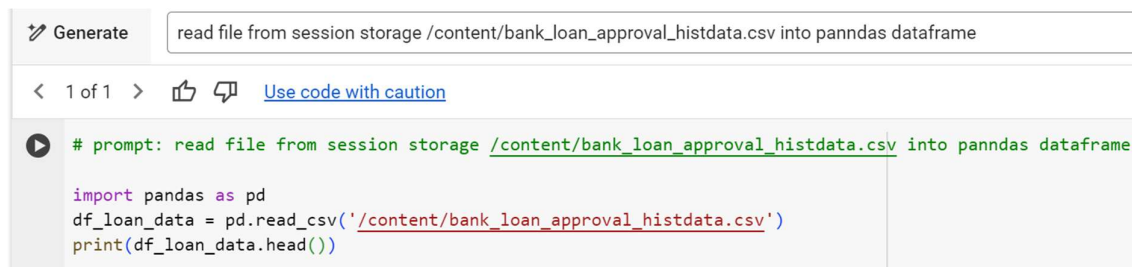
Click on the 3 dots, copy its path

The screenshot shows the "Files" sidebar in Google Colab. It displays a folder named "sample_data" and a file named "bank_loan_approval_histdata.c...". The file is circled in red. Above the file list are icons for file operations: upload, refresh, download, and delete.



Move mouse toward center of the last cell to see the +Code button. Alternatively you can click on +Code in top level menu

Generate the code as below:



When you run the cell you will see the contents of the file printed

Watch videos below for more information:

Reading csv file:

<https://www.youtube.com/watch?v=VCllZKM7Njk>

How to upload files and use them in colab

<https://youtu.be/58t0PFIWR9Y?si=2oblRNYwyl5sk1Vq>

Git

Following videos will help understand git better:

Git overview:

<https://www.youtube.com/watch?v=2ReR1YJrNOM>

Git process and commands:

https://www.youtube.com/watch?v=e9lnsKot_SQ